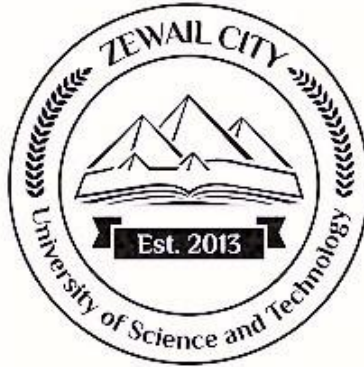


UNDERGRADUATE PROJECT
UNIVERSITY OF SCIENCE AND
TECHNOLOGY IN ZEWAIL CITY



ARM-1 8-BIT PROCESSOR
SIMULATION

Authors:

Moaz Elabd¹
Anas Refaei²
Omar Ihab Fared³
Ahmed Hany⁴
Ahmed Ghoneim⁵
Ahmed Akef⁶
Mazen Adel⁷
Abdallah Hosaam⁸
Karim Assem⁹
Mohamed Tamer¹⁰

Zeiad Hatem Salah¹¹
Mohamed Salah¹²
Youssef Mohamed¹³
Moustafa Hesham¹⁴
Shrouk Ali Hassan¹⁵
Mazen Mahmoud¹⁶
Mohamed Ali¹⁷
Abdelrahman Ayman¹⁸
Omar Yaser¹⁹
Sayed Numan²⁰

INSTRUCTOR: Prof. Elmahdy Maree

Academic year 2025/2026

All authors are Sophomore students at Communications and Information Department, UST ZC:
¹ID 202402007, ²ID 202402519, ³ID 202401437, ⁴ID 202400731, ⁵ID 202401756, ⁶ID 202401486, ⁷ID 202400692,
⁸ID 202402212, ⁹ID 202400688, ¹⁰ID 202400792, ¹¹ID 202400415, ¹²ID 202400614, ¹³ID 202400956, ¹⁴ID 202400624,
¹⁵ID 202400855, ¹⁶ID 202400447, ¹⁷ID 202400902, ¹⁸ID 202400686, ¹⁹ID 202402030, ²⁰ID 20240

Introduction

Project Overview

This report documents the design and simulation of the ARM-1 8-bit Processor, a simplified multi-cycle processor implemented and verified using the Logisim digital logic simulator.

The ARM-1 is a hypothetical, pedagogical processor modelled after the classical accumulator-based architecture. It uses a Von Neumann memory organisation in which a single 16×8 RAM serves as the combined instruction and data memory. Every instruction is executed in exactly three clock cycles — Fetch, Decode, and Execute — making the ARM-1 a representative example of a multi-cycle microarchitecture.

Processor Specifications

Property	Value
Data width	8 bits
Instruction width	8 bits (4-bit opcode + 4-bit address)
Memory	16 × 8 RAM (Von Neumann, shared instruction/data)
Architecture type	Accumulator-based
Execution model	Multi-cycle (3 states: Fetch, Decode, Execute)
Control method	Hardwired datapath (FSM ring counter + decoder)
ALU operations	Addition, Subtraction
Instruction set	LDA, ADD, SUB, STR, OUT, HLT
Output	8-bit binary display via output register O
Simulation tool	Logisim

Table 1: ARM-1 processor specifications

Overall Architecture

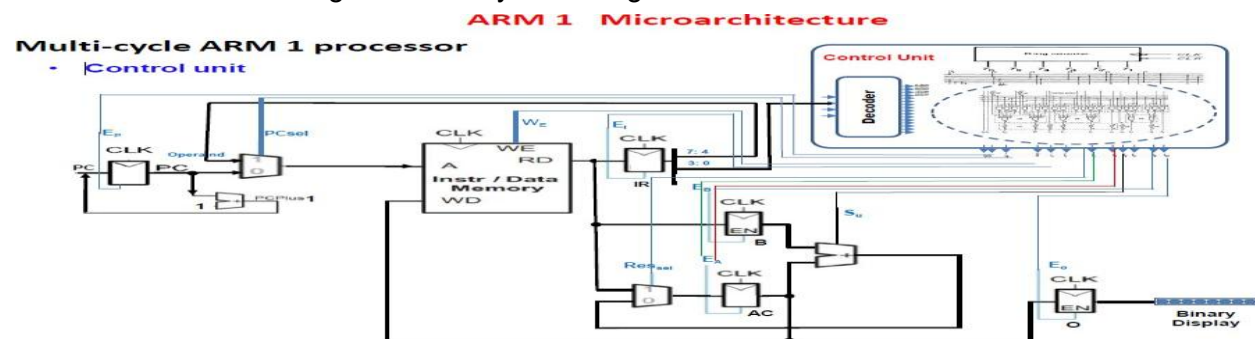
The complete ARM-1 microarchitecture consists of two interacting subsystems: the Datapath and the Control Unit.

The datapath contains all the state elements and combinational logic that manipulate data:

- **PC — Program Counter:** an 8-bit register that holds the address of the next instruction to be fetched. A +1 adder and a gating AND gate (controlled by the Cp signal) implement the auto-increment.

- **MUX (PCsel):** a 2-to-1 multiplexer that selects between the PC output and the lower nibble of the IR (IR[3:0]) as the memory address. During Fetch the PC is routed; during Decode and Execute the operand address from the IR is routed.
- **Instruction/Data Memory:** a unified 16×8 synchronous RAM with read data port (RD), write data port (WD), address port (A), and write-enable (WE).
- **IR — Instruction Register:** captures the 8-bit instruction from memory. Bits [7:4] carry the opcode; bits [3:0] carry the operand address.
- **B Register:** a temporary register that holds the memory operand during arithmetic instructions, isolating the ALU inputs from memory timing.
- **ALU:** an 8-bit adder/subtractor. The control signal $S_u = 0$ selects addition; $S_u = 1$ selects subtraction.
- **AC — Accumulator:** the primary working register, fed from either the memory read bus (for LDA) or the ALU result (for ADD/SUB), selected by Ressel.
- **O Register & Binary Display:** an output register clocked by EO, driving the 8-bit binary display.

The control unit is implemented as a hardwired finite-state machine. It contains a ring counter that cycles through the three timing states (T1 = Fetch, T2 = Decode, T3 = Execute), and a decoder that combines the current timing state with the opcode bits IR[7:4] to assert the correct combination of control signals at every clock edge.



Instruction Set Summary

The ARM-1 supports six instructions encoded in 8 bits. The upper nibble holds the opcode and the lower nibble holds a 4-bit memory address (0x0–0xF), giving direct access to all 16 memory locations.

Opcode	Mnemonic	Operation
0000	LDA addr	Load the contents of memory location addr into the accumulator: $AC \leftarrow M[\text{addr}]$
0001	ADD addr	Add memory contents to the accumulator: $AC \leftarrow AC + M[\text{addr}]$
0010	SUB addr	Subtract memory contents from the accumulator: $AC \leftarrow AC - M[\text{addr}]$
0101	STR addr	Store the accumulator to memory: $M[\text{addr}] \leftarrow AC$
1110	OUT	Transfer the accumulator to the output register: $O \leftarrow AC$
1111	HLT	Halt: stop the clock / freeze processing

CHAPTER 1:

Program Counter (PC)

1.1 Overview

The Program Counter (PC) is an 8-bit register whose responsibility is to hold the memory address of the next instruction to be fetched. At every clock edge during the Fetch state, the PC is incremented by one. During the Decode and Execute states the PC is stopped, and address control is handed over to the MUX so that the operand address carried inside the instruction can reach the memory instead.

1.2 The +1 Adder and the AND Gate

Below the PC register there is an AND gate whose two inputs are the CLK signal and the control signal C_p (= EP). The output of this AND gate gives the clock-enable (EN) pin of the PC register. By gating the clock with C_p , a rising clock edge only propagates to the register's enable input when $C_p = 1$, i.e. exclusively during the Fetch state.

The increment itself is produced by a +1 adder. This adder takes the current 8-bit PC value as input and continuously presents $PC + 1$ on its output. When the AND gate is enabled, the register loads this pre-computed value on the next clock edge.

CLK	C_p (EP)	AND output	PC behaviour
↑ (rising)	1	1	PC latches $PC+1$ (increments)
↑ (rising)	0	0	PC holds its current value
0	1	0	PC holds its current value
0	0	0	PC holds its current value

Table 1.1: AND gate truth table for the PC clock-enable

1.3 Stopping the PC During Execute — the Controlled Buffer

Once the Fetch state is complete, the control unit deasserts EP ($C_p = 0$), which freezes the PC for the remaining two states.

Without a controlled buffer, this creates a bus conflict: during Decode and Execute the register tries to hold its current value while the adder continuously presents the next value on the same input wire, potentially causing contention.

The Controlled Buffer solves this by acting as a switchable gate on the feedback path:

- **When EP = 1 (Fetch):** the buffer is open; the adder result $PC + 1$ passes through freely and is presented at the PC register input, ready to be latched on the rising clock edge.
- **When EP = 0 (Decode / Execute):** the buffer disconnects the adder output, blocking further increments until the next Fetch cycle.



PCsel	MUX Output (Address Bus)	Active State
1	IR[3:0] (operand address from instruction)	Decode / Execute

Table 2.1: PCsel multiplexer select truth table

2.1.3 What Is Inside the Memory

Two separate RAM blocks model the combined instruction and data memory. The upper RAM block holds the program (instructions), while the lower RAM block acts as a read-only program ROM. Both blocks share the same 4-bit address line driven by the PCsel MUX, and their outputs are selected through a second MUX on the read side so that only one 8-bit value appears on RD at a time, controlled by the signal Cp.

Upper RAM Block — Program Instructions

The upper RAM block stores the program that the processor executes. During the Fetch stage the MUX passes the current PC value so the processor reads the instruction and loads it into the IR via the RD line. During Execute stages that involve a memory read (LDA, ADD), the MUX instead passes IR[3:0] so the processor reads the data at the operand address. The WD and WE ports are used only during the Execute stage of STR, when the Accumulator's value must be written back to memory.

Lower RAM Block — Read-Only Program ROM

The lower RAM block acts as a read-only store. Its contents are pre-loaded before simulation begins and are never written during execution. When Cp = 0 the output MUX selects this block's output to appear on RD, providing the instruction bytes to the IR during the Fetch stage. Its WE input is permanently tied low.

The Output MUX and the Cp Signal

- **Cp = 0:** the MUX passes the lower block's output to RD — used during Fetch when the processor needs an instruction byte.
- **Cp = 1:** the MUX passes the upper block's output to RD — used during Execute when the processor needs a data value from the writable RAM.

The NOT Gate on the RAM Clock

The upper RAM block is edge-triggered. If CLK were connected directly, every rising edge would simultaneously trigger a write and cause all registers to capture their inputs — a classic write-before-stable race condition. To eliminate this hazard the RAM clock is driven through a NOT gate, so it receives $\neg\text{CLK}$. The RAM's rising edge therefore coincides with CLK's falling edge, at which point all registers have already captured their values on the previous rising edge and data lines are stable.

Stage	WE	CLK rises (registers latch)	CLK falls (RAM clock rises)
Fetch	0	IR and PC capture new values	RAM ignores (WE = 0)

Stage	WE	CLK rises (registers latch)	CLK falls (RAM clock rises)
Decode	0	No register changes	RAM ignores (WE = 0)
Execute (LDA/ADD)	0	AC captures data value	RAM ignores (WE = 0)
Execute (STR)	1	No register changes	RAM writes AC to memory

Table 2.2: NOT gate effect across the three execution stages

2.2 The Instruction Register (IR)

2.2.1 Structure and Control Signal

The IR is a simple 8-bit D flip-flop with a load enable. Its interface consists of:

- **Input** — connected directly to the memory's 8-bit RD bus.
- **Output** — the held 8-bit instruction, split into: IR[7:4] (the 4-bit opcode, fed to the control unit's decoder) and IR[3:0] (the 4-bit operand address, fed to the address MUX).
- **CLK** — the system clock.
- **Li (Load Instruction / Enable)** — the write-enable signal for the register. The register only latches a new value when Li = 1 coincides with a rising clock edge.

2.2.2 Timing Behaviour

State	Li (EI)	IR Behaviour
Fetch	1	Latches the instruction from RD on the rising clock edge
Decode	0	Holds the fetched instruction; output stable for the control unit
Execute	0	Continues to hold; IR[3:0] drives the memory address

Table 2.3: IR load-enable timing across the three execution states

2.2.3 Instruction Field Extraction

After the IR captures an instruction, the 8-bit output is passively split — no additional logic is needed:

- The upper nibble IR[7:4] is wired directly to the control unit's instruction decoder. The decoder inspects these four bits together with the current timing state (T1/T2/T3) to assert the correct control signals (EP, EI, EA, EB, EO, PCsel, WE, Ressel, Su).
- The lower nibble IR[3:0] is wired to input 1 of the address MUX. As soon as PCsel switches to 1, the operand address is immediately presented to memory without extra buffering.

2.3 Memory and IR During Each Instruction

Instruction	State	PCsel	WE	Li (EI)	Memory Role
LDA / ADD / SUB	Fetch	0	0	1	Reads instruction
	Decode	1	0	0	Reads operand data
	Execute	1	0	0	Supplies data to AC or B
STR	Fetch	0	0	1	Reads instruction
	Decode	1	0	0	Address set up
	Execute	1	1	0	Writes AC to memory
OUT / HLT	Fetch	0	0	1	Reads instruction
	Decode	—	0	0	No memory access
	Execute	—	0	0	No memory access

Table 2.4: Memory and IR control signals across all ARM-1 instructions

2.4 Registers and the ALU Datapath

The ARM-1 processor contains four registers — the Accumulator (AC), the B register, the Output register (O), and the ALU flags register. Together with the Arithmetic Logic Unit (ALU) and the Ressel MUX, these components form the execution datapath.

2.4.1 The B Register

The RD bus is shared across the entire datapath, so its value can change at any moment. If the ALU read directly from it, a bus change mid-computation would corrupt the result. The B register solves this by latching the operand when EB = 1 and holding it stable for the ALU regardless of subsequent bus changes.

When EB = 1 and CLK rises, B captures the current RD bus value. When EB = 0, B holds its previous value unchanged.

2.4.2 The ALU and ALUFlags

The ALU receives two 8-bit inputs: the current contents of AC and the current contents of B. It produces one 8-bit result presented continuously to the Ressel MUX. The ALU is purely combinational with no internal state.

The ALU also produces four condition flags — Z (Zero), N (Negative), C (Carry), V (Overflow). In the basic ARM-1 instruction set these flags are computed but not used to alter control flow, since there are no conditional branch instructions.

The Accumulator is the central working register. Every arithmetic result and every memory load is stored here. A Ressel MUX in front of the AC input selects between the RD bus (for LDA, Ressel = 0) and the ALU output (for ADD/SUB, Ressel = 1).

2.4.4 The Output Register O

Page 9 of 15

CHAPTER 3

Control Unit

3.1 Overview

The Control Unit is the “brain” of the ARM-1 processor. While the datapath contains all the hardware that manipulates data (registers, ALU, memory, buses), the control unit decides what the datapath should do at every clock cycle by asserting the correct combination of control signals. It is implemented as a hardwired finite-state machine (FSM) composed of two sub-circuits: a Ring Counter (which generates timing states) and a Decoder (which translates the current state and opcode into control signals).

Because the ARM-1 always executes every instruction in exactly three clock cycles, the FSM needs only three states: T1 (Fetch), T2 (Decode), and T3 (Execute). The control unit never needs to count higher or branch to a different number of cycles, making the ring counter an ideal — and hardware-efficient — choice.

3.2 The Ring Counter (Timing Generator)

A ring counter is a circular shift register in which exactly one bit is ‘1’ at any time and all other bits are ‘0’. In the ARM-1 a 3-bit ring counter produces three non-overlapping timing pulses:

T1	T2	T3	Processor State
1	0	0	Fetch — read instruction from memory, increment PC
0	1	0	Decode — read operand address from memory, latch B register
0	0	1	Execute — complete the instruction (ALU, write-back, store, output, halt)

Table 3.1: Ring counter states and corresponding processor states

On power-up (or after a reset), the ring counter is initialised to the state 100 (T1 = 1). On each rising clock edge the ‘1’ shifts one position to the right. After T3 the shift wraps around and returns to T1, so the sequence repeats indefinitely: T1 → T2 → T3 → T1 → ...

The HLT instruction is handled by gating the clock: when HLT is decoded in T3, the control unit stops the clock signal from reaching the ring counter (and all other registers), freezing the machine permanently in the T3 state until the circuit is reset.

3.3 The Instruction Decoder

The decoder is a combinational logic block. Its inputs are:

- **IR[7:4]** — the 4-bit opcode latched in the Instruction Register after Fetch.
- **T1, T2, T3** — the three timing outputs from the ring counter.

Its outputs are all the control signals required by the datapath:

Signal	Function
Ep (Cp)	Enables the PC to increment (AND-gated with CLK). Active only in T1.
Li (EI)	Loads the Instruction Register from the RD bus. Active only in T1.
PCsel	Selects PC (0) or IR[3:0] (1) as the memory address. Set to 1 in T2 and T3.
EB	Latches the memory operand into the B register. Active in T2.
EA	Enables loading the Accumulator. Active in T3 for LDA, ADD, SUB.
Ressel	Selects the AC input source: 0 = RD bus (LDA), 1 = ALU output (ADD/SUB).
Su	Selects ALU operation: 0 = add (ADD), 1 = subtract (SUB).
WE	Write-enables the upper RAM block. Active in T3 for STR only.
EO	Loads the Output register O from AC. Active in T3 for OUT only.
CLK gate	Stops the system clock permanently when HLT is decoded in T3.

Table 3.2: Control signals generated by the decoder

The decoder is implemented as a set of AND/OR gates (or equivalently, a ROM lookup table in Logisim). Each output is a Boolean expression of the timing state and the opcode bits. For example:

Ep = T1

(The PC increments on every Fetch cycle, regardless of opcode.)

Li = T1

(The IR is loaded on every Fetch cycle.)

PCsel = T2 | T3

(The operand address drives memory during both Decode and Execute.)

EB = T2

(The B register latches the operand during the Decode cycle.)

EA = T3 · (LDA | ADD | SUB)

(The accumulator is updated only in Execute and only for arithmetic/load instructions.)

Ressel = T3 · (ADD | SUB)

(Select the ALU output into AC for arithmetic; select RD bus for LDA.)

Su = T3 · SUB

(Select subtraction only when executing a SUB instruction.)

WE = T3 · STR

(Write to memory only in the Execute stage of a STR instruction.)

EO = T3 · OUT

(Load the output register only in the Execute stage of an OUT instruction.)

3.4 State-by-State Signal Table

The table below consolidates all control signals for every combination of timing state and instruction, providing a complete specification of the control unit's behaviour:

State	Ep (Cp)	Li (EI)	PCsel	EB	EA/Ressel/Su/EO/WE	Next
T1 Fetch	1	1	0	0	All = 0	T2
T2 Decode	0	0	1	1	EB=1, others=0	T3
T3 LDA	0	0	1	0	EA=1, Ressel=0	T1
T3 ADD	0	0	1	0	EA=1, Ressel=1, Su=0	T1
T3 SUB	0	0	1	0	EA=1, Ressel=1, Su=1	T1
T3 STR	0	0	1	0	WE=1	T1
T3 OUT	0	0	0	0	EO=1	T1
T3 HLT	0	0	0	0	CLK stopped	—

Table 3.3: Complete control signal truth table for all instructions and states

3.5 Execution Walk-Through: LDA followed by ADD

To make the control unit concrete, consider the short program:

```
Address 00h: 0000 0101 → LDA 05h    (load M[5] into AC)
Address 01h: 0001 0110 → ADD 06h    (add M[6] to AC)
Address 02h: 1110 0000 → OUT         (display AC)
Address 03h: 1111 0000 → HLT         (stop)
```

Cycles 1–3 — LDA 05h:

1. **T1 (Fetch):** Ep=1, Li=1, PCsel=0. Memory[00h] = 0000 0101 is placed on RD. IR latches 0000 0101. PC becomes 01h.
2. **T2 (Decode):** PCsel=1, EB=1. IR[3:0] = 0101 = 05h. Memory[05h] is placed on RD. B latches the value at M[05h].
3. **T3 (Execute):** EA=1, Ressel=0. The Ressel MUX selects the RD bus. AC latches M[05h] via the pulse generator.

Cycles 4–6 — ADD 06h:

4. **T1 (Fetch):** Ep=1, Li=1, PCsel=0. Memory[01h] = 0001 0110 is placed on RD. IR latches 0001 0110. PC becomes 02h.
5. **T2 (Decode):** PCsel=1, EB=1. IR[3:0] = 0110 = 06h. Memory[06h] is placed on RD. B latches the value at M[06h].
6. **T3 (Execute):** EA=1, Ressel=1, Su=0. The ALU computes AC + B. The Ressel MUX selects the ALU output. AC latches the sum.

3.6 Logisim Implementation of the Control Unit

In Logisim the control unit is built from two sub-circuits placed side by side:

Ring Counter Sub-circuit

Three D flip-flops are connected in a circular chain. The Q output of each flip-flop feeds the D input of the next, and the Q output of the last flip-flop feeds the D input of the first. All three flip-flops share the same CLK input (which may be gated by the HLT logic). A reset line forces the initial state to 100 so that T1 is always the first active state.

Decoder Sub-circuit

The decoder is a two-level combinational network. The first level consists of AND gates that compute the product terms (e.g., T3 AND LDA, T3 AND ADD, etc.). The second level consists of OR gates that combine the product terms into each output signal. In Logisim this can be implemented directly with logic gates or with a ROM whose address is the concatenation of the opcode bits IR[7:4] and the timing bits T3:T1, and whose data output word contains all control signal bits.

The complete control unit fits on a single Logisim sheet and interfaces with the rest of the datapath via the nine control wires listed in Table 3.2 above.

3.7 Handling the HLT Instruction

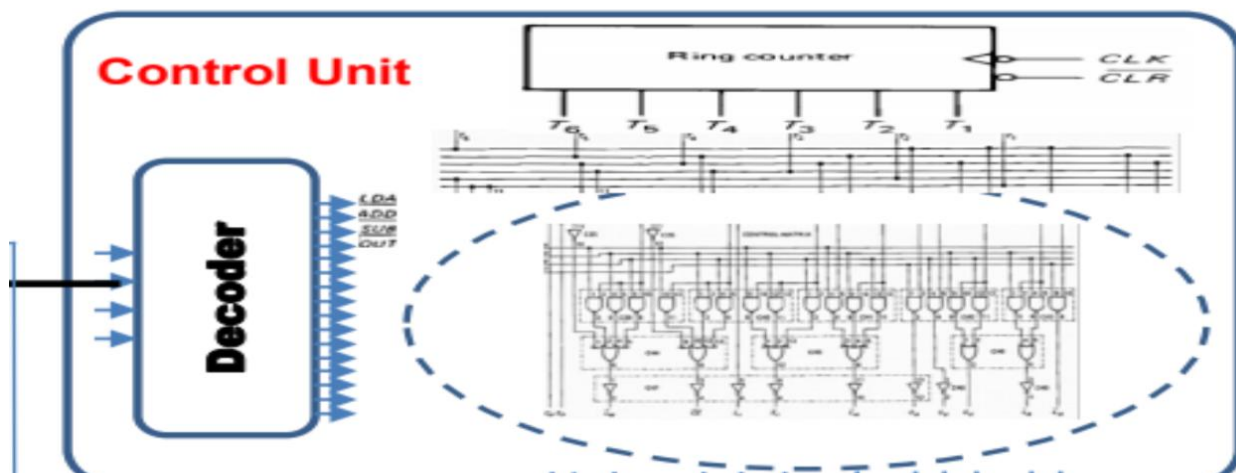
The HLT instruction (opcode 1111) is handled specially. When the decoder detects $IR[7:4] = 1111$ and $T_3 = 1$, it asserts a “Halt” signal that is ANDed with the CLK line before it reaches the ring counter and all the datapath registers. Once this gate closes, no rising clock edge can propagate, so every register (including the ring counter) freezes in place. The processor can only resume if an external reset is applied.

3.8 Summary

The ARM-1 control unit is a minimal yet complete hardwired FSM. Its two-component design — a 3-bit ring counter plus a combinational decoder — generates all nine control signals needed to sequence Fetch, Decode, and Execute across the six instructions of the ARM-1 instruction set. The approach is deterministic, has zero idle cycles, and requires no microcode ROM or complex sequencing logic, making it an ideal teaching example of hardwired control.

Instruction	Cycles	Ep	Li	EA	WE / EO
LDA addr	3	T1	T1	T3	—
ADD addr	3	T1	T1	T3	—
SUB addr	3	T1	T1	T3	—
STR addr	3	T1	T1	—	WE=T3
OUT	3	T1	T1	—	EO=T3
HLT	3	T1	T1	—	CLK gated

Table 3.4: Instruction cycle summary



Conclusion

This report has presented a complete description of the ARM-1 8-bit multi-cycle processor: its datapath components (Program Counter, unified Instruction/Data Memory, Instruction Register, B register, ALU, Accumulator, Output register) and its Control Unit (ring counter timing generator and combinational instruction decoder).

The ARM-1 demonstrates several key principles of digital processor design: the use of a shared memory with a multiplexed address bus to reduce hardware cost, the importance of intermediate registers (IR, B) to isolate timing-sensitive signals, the use of a pulse generator to prevent multi-latching in the accumulator, and the elegance of a hardwired FSM as a control strategy when the instruction set is small and fixed-length.

The three-cycle Fetch–Decode–Execute model is a foundational concept in computer architecture, directly generalising to the multi-stage pipelines used in modern processors. By implementing and simulating the ARM-1 in Logisim, the design team gained hands-on experience with every level of the processor design hierarchy: from individual flip-flops and gates up to a complete, functional stored-program computer.